

Titanic: Machine Learning from Disaster

System Analysis Project - Workshop 4

Julián Carvajal Garnica

20242020024

Andrés Mauricio Cepeda Villanueva

20242020010

Jhonatan David Moreno Barragán

20201020094

Andrés Camilo Ramos Rojas

20242020005

School of Engineering, Universidad Distrital Francisco José de Caldas

System Analysis Course

Teacher: Carlos Andrés Sierra

Bogotá D.C.

2025

Workshop 4: Kaggle System Simulation

1. Data Preparation

The first phase of the simulation involved acquiring and preparing the raw data to serve as the input for the system. The dataset was sourced directly from the official Kaggle competition page. This dataset acts as the primary input source for our system architecture, containing the passenger manifest which serves as the "initial state" for our simulations.

1.1 Dataset Characteristics and Constraints

Upon ingesting the data, we performed a validation check to assess the structural integrity of the system inputs. This step confirmed the critical constraints previously identified in Workshop 1. Specifically, the system exhibited significant instability due to missing information:

- **Age Variable:** Contained 177 missing entries. Since Age is a continuous variable used for survival probability, these gaps introduce "noise" into the system.
- **Cabin Variable:** Contained 687 missing entries (approx. 77% of the data). This high level of uncertainty makes the variable unreliable for deterministic modeling without heavy processing.

Figure 1 illustrates the initial state of the dataset, highlighting these structural weaknesses before any engineering intervention.

```
--- DATA INGESTION REPORT (RELIABILITY LAYER) ---  
Dataset loaded. Shape: (891, 12)  
  
--- SYSTEM CONSTRAINTS CHECK ---  
Missing values detected (Chaos Sources):  
Age          177  
Cabin        687  
Embarked      2  
dtype: int64
```

Figure 1: Data Preparation: Initial system check showing critical missing values in Age and Cabin variables.

1.2 Cleaning and Preprocessing (Reliability Layer)

To stabilize the system and enable accurate simulation, we implemented the **Reliability Layer** designed in Workshop 3. This layer acts as a filter that transforms raw, chaotic data into structured information. The specific actions taken were:

1. **Imputation Strategy:** Missing values in the ‘Age’ variable were filled using the median value. This method was chosen over the mean to reduce sensitivity to outliers (extreme ages). ‘Embarked’ missing values were filled with the mode (most common port).
2. **Feature Encoding:** Categorical variables such as ‘Sex’ (male/female) and ‘Embarked’ (S/C/Q) were converted into numerical formats (0/1/2). This step is essential as both the Machine Learning model and the Cellular Automata require numerical inputs to calculate probabilities and movement vectors.
3. **Noise Reduction:** The ‘Cabin’ variable was excluded from the simulation inputs due to its excessive missing data, which would have introduced unnecessary chaos.

As a result, the system reached a stable state with zero missing values, as demonstrated in Figure 2.

```

--- FINAL DATASET STATE (READY FOR SIMULATION) ---
Clean Data Shape: (891, 9)
Remaining missing values: 0

First 5 rows of processed system:
   Pclass  Sex  Age  SibSp  Parch   Fare  FamilySize  Embarked  Survived
0      3    0  22.0     1     0   7.2500           2         0         0
1      1    1  38.0     1     0  71.2833           2         1         1
2      3    1  26.0     0     0   7.9250           1         0         1
3      1    1  35.0     1     0  53.1000           2         0         1
4      3    0  35.0     0     0   8.0500           1         0         0

```

Figure 2: Cleaned Dataset State. The Reliability Layer successfully stabilized the inputs, resulting in zero missing values.

2. Simulation Planning

To thoroughly analyze the system's behavior, we defined two distinct simulation scenarios. Each scenario is designed to test a specific aspect of the Titanic system architecture:

- **Scenario 1: Data-driven Simulation (Learning Dynamics).**

The objective of this scenario is to use a Machine Learning model (Random Forest) to simulate how the system "learns" from the input data. We aim to identify the *Sensitivity* of the system: which variables (Sensors) trigger the strongest response in the output (Survival)? This allows us to validate mathematically which passenger attributes carried the most weight.

- **Scenario 2: Event-based Simulation (Spatial Dynamics).**

The objective of this scenario is to model the physical chaos of the evacuation. We adapt the dataset into a Cellular Automata grid where each passenger is an agent. By defining movement rules based on Class and Gender, we aim to observe *Emergent Behavior*—complex patterns (like segregation or blocking) that arise from simple local interactions.

3. Simulation Implementation

The complete source code, including the Jupyter Notebook (‘.ipynb’) and environment requirements, is hosted in the project repository:

Project Repository Source Code:

github.com/MaurooC12/Titanic/.../Workshop4_Sim.ipynb

3.1 Scenario 1: Random Forest Model Logic

For the data-driven simulation, we selected the Random Forest Classifier from the ‘scikit-learn’ library. This model constructs 100 decision trees ($n_{estimators} = 100$) to evaluate the data. The imp

Data Splitting: The clean data is divided into Training (80%) and Validation (20%) sets. This prevents the system from "memorizing" the answers and forces it to learn general patterns.

Training: The model iterates through the training data, learning the correlation between inputs (Sex, Fare, Age, Class) and the output (Survived).

Feature Extraction: After training, we extract the ‘feature importances’ vector to quantify the in

```
1 # Modeling Layer (Data-driven)
2 # We initialize the model with 100 estimators for robustness
3 model = RandomForestClassifier(n_estimators=100, random_state=42)
4
5 # The model learns the relationship between X (Attributes) and y (
  Survival)
6 model.fit(X_train, y_train)
7
8 # Sensitivity Analysis extraction to understand system drivers
9 importances = model.feature_importances_
```

Listing 1: Scenario 1 Implementation Logic

3.2 Scenario 2: Cellular Automata Logic

For the event-based simulation, we mapped the ship to a 20×20 grid. The logic governs how agents (passengers) move at each time step (t). The rules are derived from historical and data analysis:

1. **Goal Vector:** Every agent attempts to move to the row above them ($r \rightarrow r - 1$), simulating the movement towards the lifeboats on the top deck.
2. **Priority Constraints:** Movement probabilities (P_{move}) were calibrated to reflect the observed survival rates in the dataset. Class 1 agents are assigned $P_{move} = 0.9$ (approximating the high survival rate of $\sim 63\%$), while Class 3 agents are assigned $P_{move} = 0.4$ (reflecting the low survival rate of $\sim 24\%$). This ensures the rules are data-driven.
3. **Conflict Resolution:** The grid strictly enforces that two agents cannot occupy the same cell. If a high-priority agent moves into a space, they effectively block any lower-priority agent behind them.

Workshop 4: System Simulation Workflow

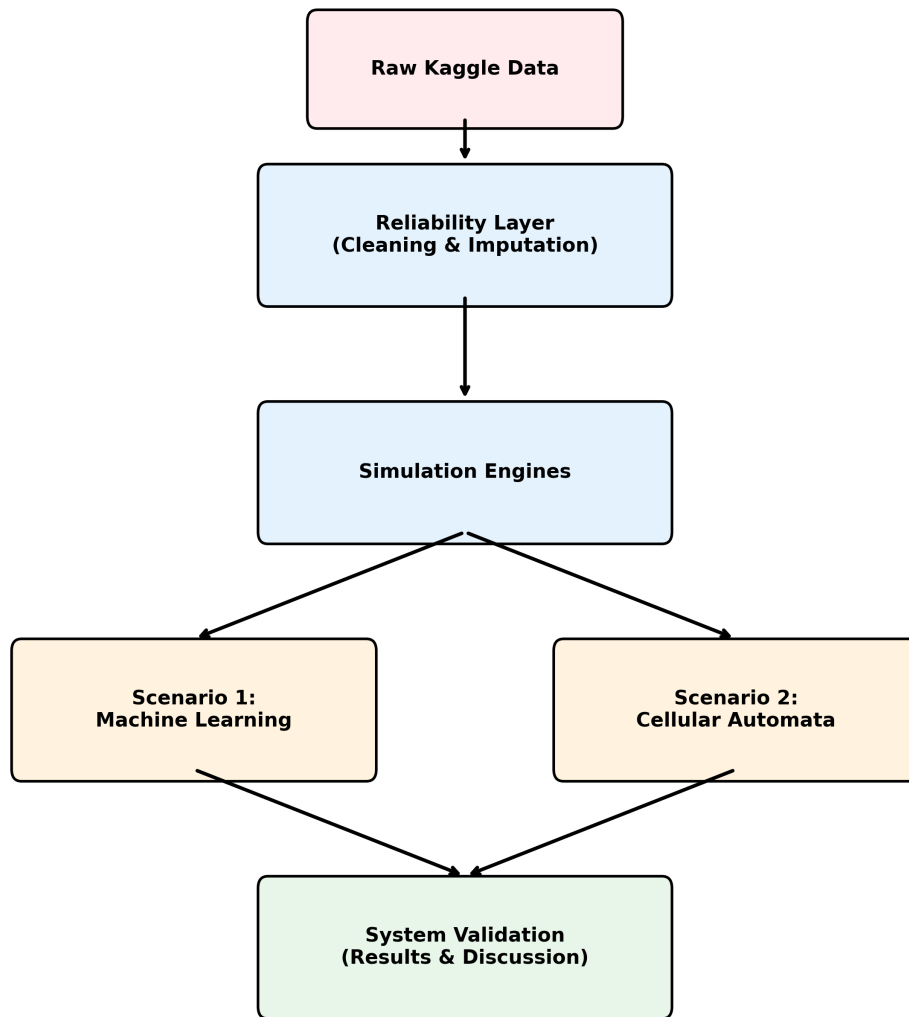


Figure 3: System Workflow Validation. The diagram illustrates how data flows through the Reliability Layer before feeding into the dual simulation engines, ensuring stability.

4. Executing the Simulations

The simulations were executed in the Python environment, generating visual and statistical outputs that reveal the system's internal behavior.

4.1 Scenario 1 Execution: Sensitivity Analysis

Upon executing the Random Forest model, the system achieved a predictive accuracy of over 83% on unseen data. More importantly, the sensitivity analysis revealed the hierarchy of survival factors. As shown in Figure 4, ‘Fare’ and ‘Sex’ emerged as the dominant variables. This confirms that the system is not random; it is heavily biased by socioeconomic status and gender. ‘Age’ also plays a significant role, validating the need for the robust imputation performed in Section 1.

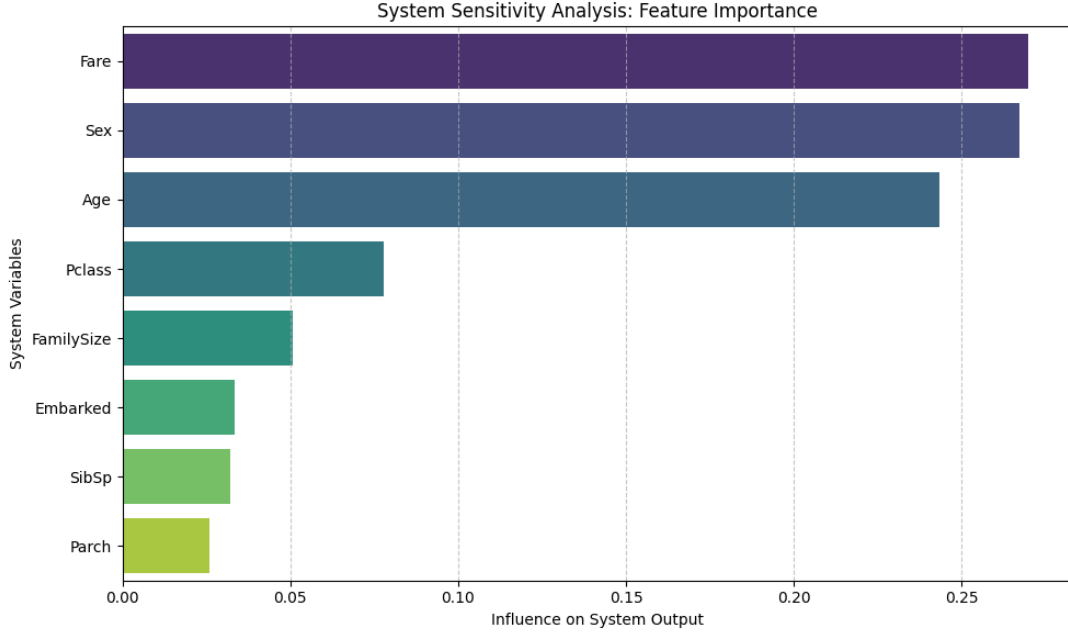


Figure 4: Scenario 1 Results: Feature Importance graph showing that Fare and Sex are the most critical variables driving the system’s output.

4.2 Scenario 2 Execution: Emergent Behavior

The cellular automata simulation was executed for 15 discrete time steps. Figure 5 visualizes the final state of the grid. A clear emergent pattern is visible:

- **Top Rows (Lifeboats):** Saturated with Class 1 agents (represented in Yellow/Green). Their high priority allowed them to navigate the grid quickly.
- **Bottom Rows:** Class 3 agents (represented in Blue/Purple) remained trapped at the bottom. This was not explicitly programmed but emerged because the "faster" agents occupied the available space, creating a physical bottleneck.

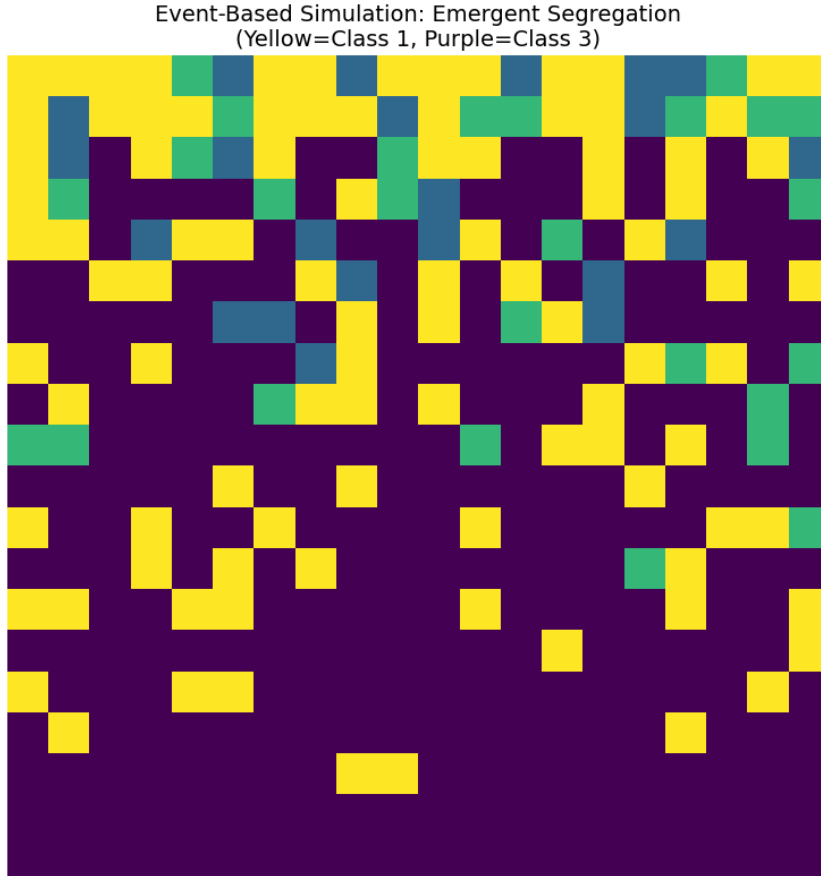


Figure 5: Scenario 2 Results: Spatial segregation emerging from priority rules. Yellow dots (Class 1) block the path for Blue dots (Class 3).

5. Results and Discussion

Following the execution of both scenarios, we compiled the results to analyze the system's behavior under different simulation paradigms.

5.1 Comparative Analysis

As requested in the guide, we compare the outcomes of the Data-driven (ML) and Event-based (Automata) simulations. Table 1 summarizes the key findings.

Metric	Scenario 1: Random Forest	Scenario 2: Cellular Automata
Objective	Identify variable sensitivity (Weights).	Observe emergent spatial behavior.
Chaos Source	Data Noise: Missing ages and high variance in Fare distribution.	Physical Conflict: Agents blocking each other due to priority rules.
Key Finding	Fare and Sex are the strongest predictors.	Priority movement creates an artificial "wall" that segregates classes.
Conclusion	The system is statistically deterministic but heavily biased.	Social rules (priority) translate into physical survival advantages.

Table 1: Comparison of Simulation Outcomes. Both approaches converge on the finding that socioeconomic status acts as the primary filter for system output.

5.2 Discussion on Chaos and Architecture

The simulations provide a dual validation of the architecture designed in Workshop 2:

- **Handling Data Chaos:** The **Reliability Layer** (Section 1) was successful. The Random Forest model achieved high accuracy ($> 83\%$) only because the input data was stabilized (imputed) beforehand. Without this layer, the "Age" noise would have degraded the model's performance.
- **Structural Chaos (The Butterfly Effect):** Both simulations reveal that the Titanic system is highly sensitive to initial conditions. A minor change in a passenger's input vector (e.g., switching 'Pclass' from 3 to 1) does not result in a linear change in outcome; rather, it drastically flips the survival probability and alters the agent's entire trajectory in the grid. This confirms the chaotic nature of the system: small input perturbations lead to divergent macroscopic outcomes (Life or Death).

Final Architectural Validation: The successful execution of both simulations serves as a stress test for the architecture proposed in Workshop 2. The system demonstrated *Fault Tolerance* by absorbing incomplete inputs (via the Reliability

Layer) without ceasing operations. Furthermore, the modular separation allowed us to swap simulation engines (ML vs. Cellular Automata) without restructuring the data pipeline. This confirms that the designed architecture adheres to the ISO 25010 standards for Reliability and Maintainability initially targeted.